CHAPTER 1

INTRODUCTION

1.1 PERSCPECTIVE

The purpose is to introduce the UFC Management System, a standalone desktop application designed for efficient management of Ultimate Fighting Championship fighter information, match scheduling, and ranking calculations. The UFC Management System is a comprehensive solution responsible for providing centralized data management to administrators and users involved in UFC event organization.

The application workflow begins with user authentication. Users must first create an account through the registration page, where they provide username, email, and password. After registration, accounts require admin approval before users can access the system. Once approved, users can log in with their credentials.

The application maintains two distinct user roles:

- Admin - Managing fighter profiles (add, edit, delete fighters) ,Scheduling and managing matches,Verifying and approving user registrations,Managing UFC events,Viewing system analytics and reports ,Calculating and updating P4P (Pound-for-Pound) rankings

- User - Searching for fighter information,Viewing match schedules and details,Accessing P4P rankings,Viewing their personal profile,Accessing UFC news and updates

1.2 OBJECTIVE

The only main objective is to Provide a unified platform for storing and managing all fighter-related information, match records, and event details in a structured database and Implement automatic P4P (Pound-for-Pound) ranking calculations based on fighter performance metrics including win ratio, knockout percentage, and fight history. It provides comprehensive analytics dashboard showing system metrics including total fighters, matches, pending users, and active events.

It enables admins to schedule matches, assign fighters, record results, and maintain complete match history also maintains data consistency and integrity through validation checks, error handling, and proper database transaction management. It encompass providing comprehensive dashboards that display system metrics including fighter counts, match volumes, pending user registrations, and active events, enabling data-driven administrative decisions and performance monitoring.

1.3 SCOPE

The scope of the project is to reduce manual data entry errors through validation And provide instant access to fighter statistics and match history ,also automates complex ranking calculations by centralizing ufc data in a secure, reliable database and Complete fighter profile management with detailed statistics , Match scheduling and result recording and an automated P4P ranking calculation algorithm , User authentication with admin approval workflow.

The current implementation scope includes a fully functional desktop application compatible with Windows, macOS, and Linux platforms through Java's cross-platform capabilities.The system delivers complete fighter profile management capabilities, capturing detailed statistics including name, nationality, age, physical attributes, weight class, and comprehensive fight records.

CHAPTER 2

REQUIREMENT DESCRIPTION

2.1 FUNCTIONAL REQUIREMENTS

The functional requirements for the UFC Management System describe the operations and capabilities available in the system:

- The system provides functionality for user registration with username, email, and password
- The system provides functionality to add new fighter profiles with details: name, nationality, age, weight class, height, reach, record (wins-losses-draws)
- The system allows recording of match results (winner, method, time)
- The system provides functionality to search fighters by name with detailed fighter profiles and also to show current P4P rankings
- The system provides detailed fighter profiles , match profiles , event profiles with complete statistics

2.2 NON-FUNCTIONAL REQUIREMENTS

The non-functional requirements describe the quality attributes, constraints, and platform requirements for the UFC Management System:

- The system shall load the landing page within 2 seconds of application launch
- The application must be user friendly and must be very interactive to the user.
- The system shall maintain data integrity through proper transaction management and implements error handling to prevent application crashes
- The application shall run on any system with Java Runtime Environment (JRE) 8 or higher

CHAPTER 3

SYSTEM DESIGN

3.1 ARCHITECTURE DESIGN

Architectural diagram implies the flow of the system. The flow starts whether the user has account or not. If the user had an account it directly takes to the login page and after he logged into the system, it enables the user to donate the food for which the user should entry the count so the count of the people in receiver which is closer to the data entered by the user will be displayed to the donor.

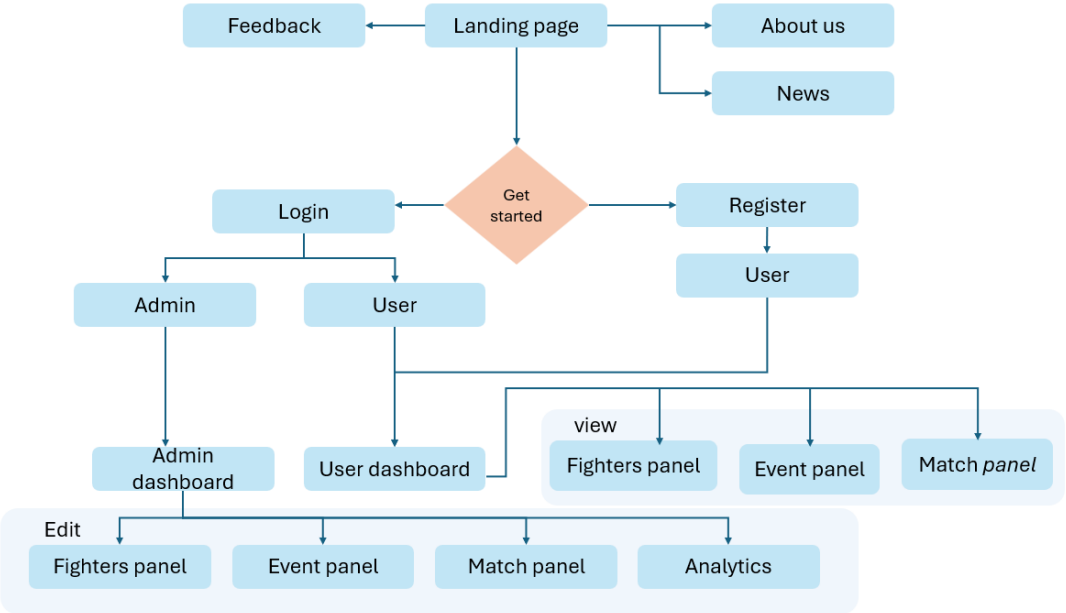

Figure 3.1: Architecture Diagram of UFC Management System

3.2 DESIGN COMPONENTS

3.2.1 Front End:

The UFC Management System uses Java Swing framework for developing interactive graphical user interfaces.

3.2.2 Back End:

The system uses Java for backend logic and business operations

3.3 DATABASE DESCRIPTION

Listed below gives a description of database document schemas used for UFC Management System.

3.3.1 User Table Structure

As shown in Table 3.1, the users table contains authentication and user management information. **Table 3.1: User Table Description**

Attribute Name	Type	Width	Constraint(s)	Description
username	VARCHAR	50	PRIMARY KEY	username for login
password	VARCHAR	100		User password
email	VARCHAR	100		User email address
user_type	VARCHAR	20		User role
created_date	DATE			creation timestamp

3.3.2 Fighter Table Structure

As shown in Table 3.2, the fighters table contains detailed fighter profile information.

Table 3.2: Fighter Table Description

Attribute Name	Type	Width	Constraint(s)	Description
fighter_id	NUMBER		PRIMARY KEY	Auto-generated unique fighter ID
name	VARCHAR	100		Fighter's full name
nationality	VARCHAR	50		Fighter's country
age	NUMBER			Fighter's age
weight_class	VARCHAR	30		Weight division
height_cm	NUMBER			Height in centimeters
reach_cm	NUMBER			Reach in centimeters
wins	NUMBER			Total wins
losses	NUMBER			Total losses
draws	NUMBER			Total draws
knockouts	NUMBER			Knockout wins

3.3.3 Match Table Structure

As shown in Table 3.3, the matches table stores match scheduling and results.

Table 3.3: Match Table Description

Attribute Name	Type	Width	Constraint(s)	Description
match_id	NUMBER		PRIMARY KEY	match ID
fighter1_id	NUMBER		FOREIGN KEY	First fighter ID
fighter2_id	NUMBER		FOREIGN KEY	Second fighter ID
match_date	DATE			match date
location	VARCHAR	100		Match venue
weight_class	VARCHAR	30		Weight division
total_rounds	NUMBER			Number of rounds
winner_id	NUMBER		FOREIGN KEY	Winning fighter ID
finish_method	VARCHAR	50		KO, Submission
finish_round	NUMBER			Round where fight ended

finish_time	VARCHAR	10		Time in round (MM:SS)

3.3.4 Event Table Structure

As shown in Table 3.4, the events table manages UFC event information.

Table 3.4: Event Table Description

Attribute Name	Type	Width	Constraint(s)	Description
event_id	NUMBER		PRIMARY KEY	Auto-generated unique event ID
event_name	VARCHAR	100		Event title
event_date	DATE			Event date
location	VARCHAR	100		Event venue
description	VARCHAR	500		Event details
status	VARCHAR	20		Event status
created_date	DATE			Event creation timestamp

3.4 LOW LEVEL DESIGN

The following section illustrates the functionalities of the system including authentication, fighter management, and match scheduling

3.4.1 Login

Table 3.4 shows the login details of the application.

Table 3.4 Login Details

Files used	ui.pages.LoginPage.java, services.AuthService.java
Short Description	Allows users to login to the application with username and password authentication
Arguments	username (String), password (String)
Return	User type: "admin", "user", "pending", or null (invalid)
Pre-Condition	User must have registered account in database
Post-Condition	If admin: Admin Dashboard displayed; If user: User Dashboard displayed; If pending: Pending message shown
Exception	Invalid credentials, Database connection failure

3.4.2 Add Fighter

Table 3.5 shows the add fighter functionality details.

Table 3.5: Add Fighter Function Details

Files used	ui.pages.ManageFightersPage.java, services.FighterService.java
Short Description	Allows admin to add new fighter profile to database
Arguments	name, nationality, age, weight_class, height, reach, wins, losses, draws, knockouts

Return	Boolean (success/failure)
Pre-Condition	User must be logged in as admin
Post-Condition	New fighter added to database, fighter list refreshed
Exception	Duplicate fighter name, Invalid data format, Database error
Actor	Admin

3.4.3 Schedule Match

Table 3.7 shows the match scheduling functionality details.

Table 3.7: Schedule Match Function Details

Files used	ui.pages.ManageMatchesPage.java, services.MatchService.java
Short Description	Allows admin to schedule new match between two fighters
Arguments	fighter1_id, fighter2_id, match_date, location, weight_class, total_rounds
Return	Boolean (success/failure)
Pre-Condition	User must be logged in as admin, Both fighters must exist in database
Post-Condition	New match scheduled in database
Exception	Invalid fighter IDs, Past date selection, Database error
Actor	Admin

3.5 USER INTERFACE DESIGN

3.5.1 Landing Page

Figure 3.2 provide the interface for landing page.

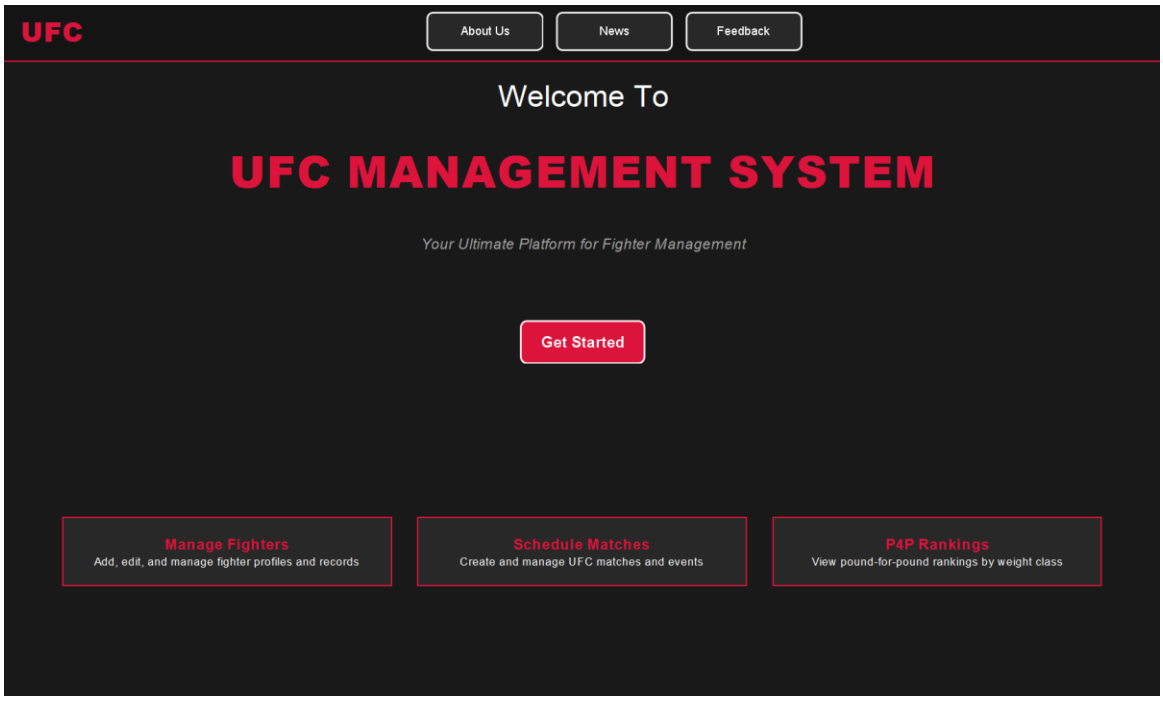

Figure 3.2: Landing Page of UFC Management System

3.5.4Login page

Figure 3.6 provide the interface for Login page

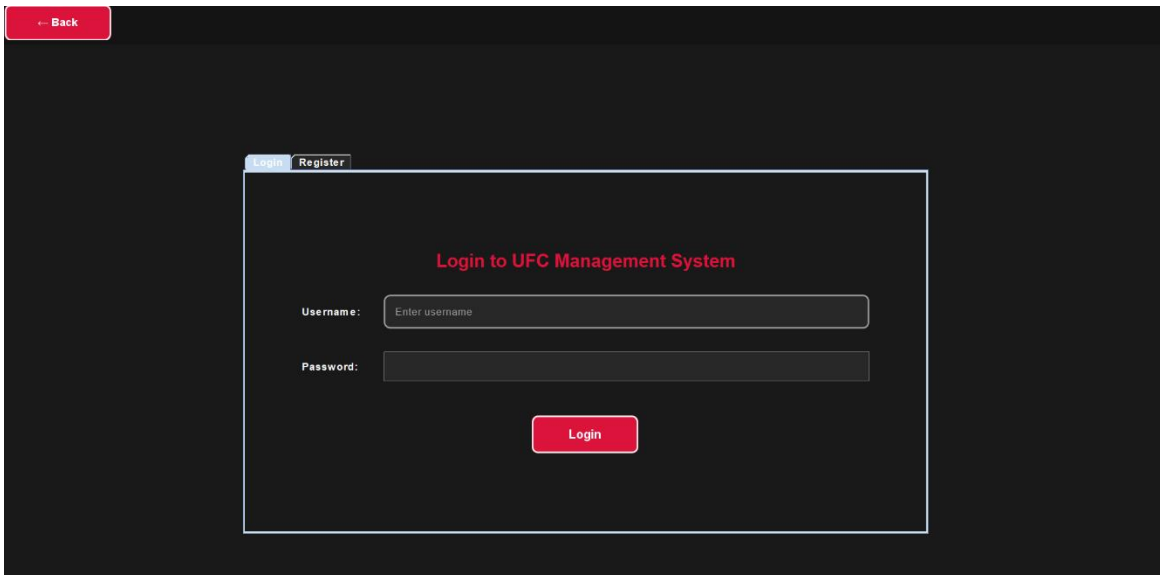

Figure 3.6: Login Page

3.5.5Registration page

Figure 3.7 provide the interface for Registration page

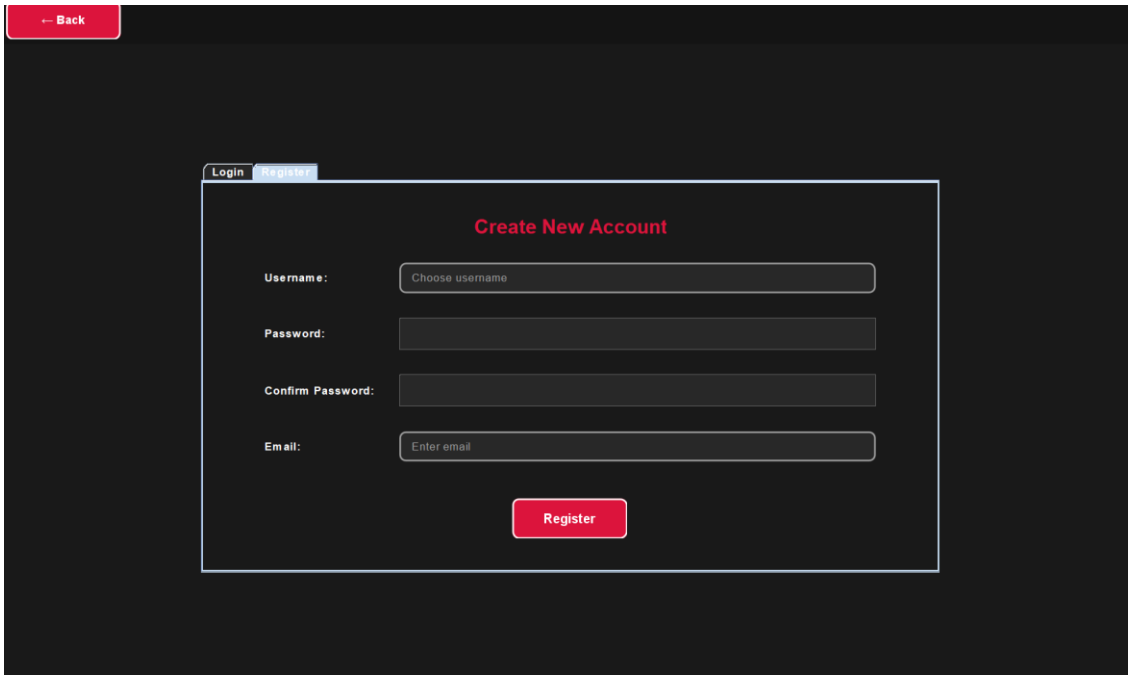

Figure 3.7: Registration Page

3.5.3 Admin Dashboard

Figure 3.4 provides the interface for Admin Dashboard with all administrative features

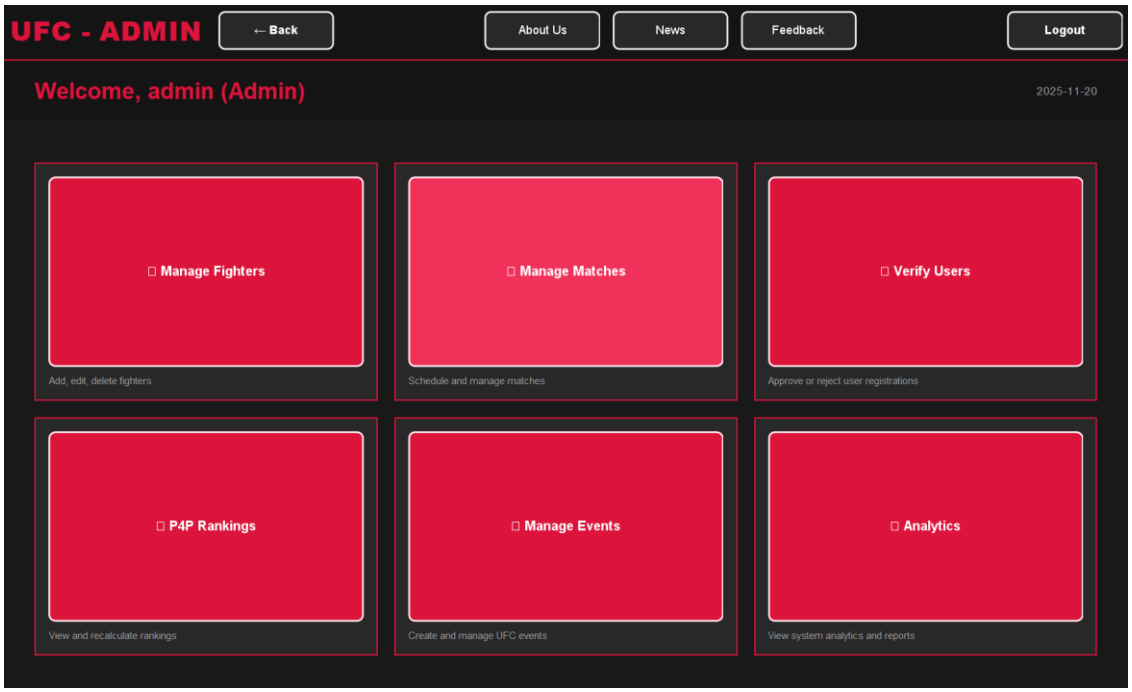

Figure 3.8: Admin Dashboard

3.5.3 News Page

Figure 3.5 provides the interface for the News Page

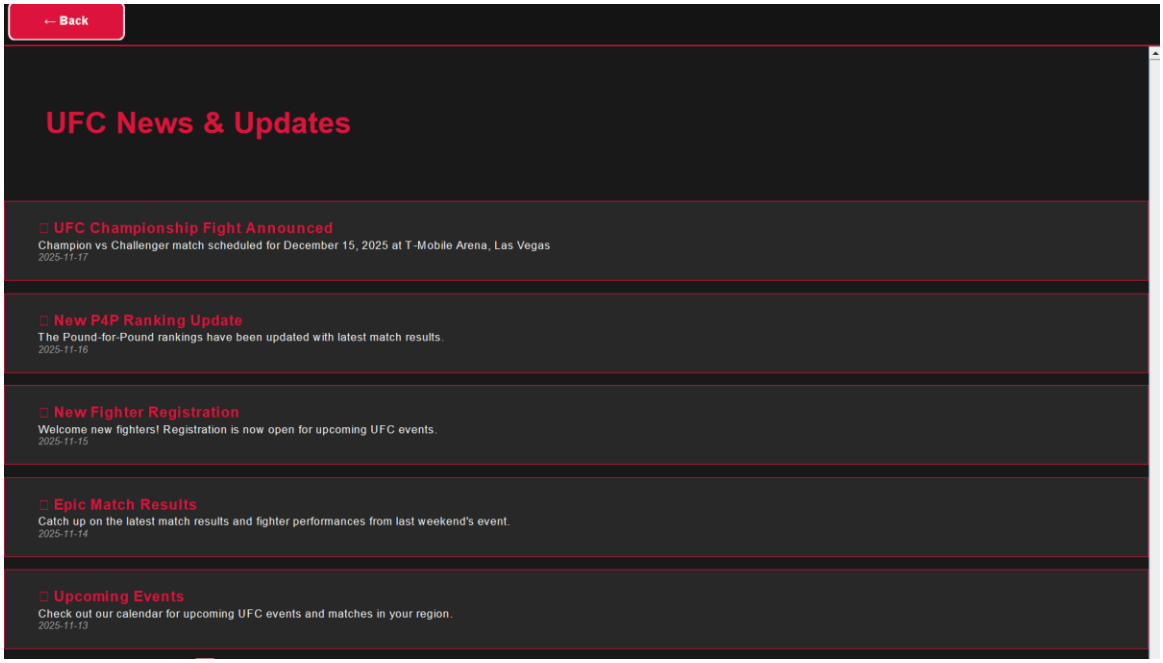

Figure 3.9: News Page

CHAPTER 4

SYSTEM IMPLEMENTATION

4.1LOGIN IMPLEMENTATION

The login credentials are obtained. If the credentials are OK, then the user is redirected to the homepage

```
IF user_type == "admin"
    OPEN AdminDashboard
    CLOSE LoginPage
    RETURN success
ELSE IF user_type == "user"
    OPEN UserDashboard
    CLOSE LoginPage
    RETURN success
ELSE IF user_type == "pending"
    SHOW MESSAGE "Account pending admin approval"
    RETURN pending
END IF
```

4.2SIGNUP IMPLEMENTATION

The form fields are obtained. If they are valid, then the user is added to the database.

```
CHECK database: SELECT COUNT(*) FROM users WHERE username = ?
IF count > 0
```

SHOW ERROR "Username already exists"

RETURN failure

END IF

CHECK database: SELECT COUNT(*) FROM users WHERE email = ?

IF count > 0

SHOW ERROR "Email already registered"

RETURN failure

END IF

IF all validations pass

INSERT INTO users (username, email, password, user_type, created_date)

VALUES (?, ?, ?, 'pending', SYSDATE)

SHOW SUCCESS "Registration successful. Awaiting admin approval."

RETURN success

ELSE

SHOW ERROR "Please enter valid details"

RETURN failure

END IF

4.3FIGHTER IMPLEMENTATION

CHECK database: SELECT COUNT(*) FROM fighters WHERE LOWER(name) = LOWER(?)

IF count > 0

SHOW ERROR "Fighter already exists"

```
RETURN failure

END IF

CALCULATE win_ratio = wins / (wins + losses + draws)

CALCULATE knockout_percentage = (knockouts / wins) * 100 IF wins > 0 ELSE
0

IF all validations pass

GENERATE fighter_id = NEXT_VALUE(fighter_id_seq)

INSERT INTO fighters (fighter_id, name, nationality, age, weight_class,

                height_cm, reach_cm, wins, losses, draws, knockouts, created_date)

VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, SYSDATE)


SHOW SUCCESS "Fighter added successfully"

REFRESH fighter list

RETURN success


ELSE

SHOW ERROR "Please enter valid fighter details"

RETURN failure

END IF
```

CHAPTER 5

RESULTS AND DISCUSSION

5.1 TEST CASES AND RESULTS

5.1.1 Test Cases and Results for Login function:

The Table 5.1, Table 5.2 shows that the possible test data for the both positive and negative test case given below, if the user is already having account then the output is true otherwise false.

Table 5.1: Positive Test Case and result for Login

Test Case Description	Tests whether valid admin credentials allow successful login and redirect to Admin Dashboard
Test Data	Username: admin, Password: admin123
Expected Output	TRUE - Redirect to Admin Dashboard
Actual Output	TRUE - Admin Dashboard opened successfully
Result	PASS

Table 5.2: Negative Test Case and result for Login

Test Case Description	Tests whether invalid credentials are properly rejected
Test Data	Username: testuser, Password: wrongpassword
Expected Output	FALSE - Error message "Invalid username or password"
Actual Output	FALSE - Error message displayed correctly
Result	PASS

CHAPTER 6

CONCLUSION AND FUTURE ENHANCEMENT(S)

The UFC Management System is a comprehensive desktop application successfully designed and implemented to manage fighter information, match scheduling, P4P rankings, and user authentication for Ultimate Fighting Championship events. The system addresses the critical need for centralized data management in combat sports administration. The dual-role system (Admin and User) effectively segregates responsibilities, ensuring secure administrative operations while providing appropriate access to regular users. The system successfully implements all core functionalities including fighter management, match scheduling, automated ranking calculations, and secure user authentication . In future , the system can be a real time match updating system by integrating it into a web based or application with sports APIs.

APPENDIX – A

SYSTEM REQUIREMENTS

HARDWARE REQUIREMENT:

- Processor: Intel Core i3
- RAM: 4 GB
- Hard Disk: 500 MB free space (application + database)
- Network: Not required (standalone application)

SOFTWARE REQUIREMENT:

Operating System :	Any
DBMS :	Oracle Express 21c Edition
IDE used :	Android Studio

APPENDIX – B

SOURCE CODE

User.java

```
package models;


import java.sql.Timestamp;


public class User {
    private int userId;
    private String username;
    private String password;
    private String email;
    private String userType;  // 'admin' or 'user'
    private String status;    // 'active', 'blocked', 'pending'
    private Timestamp createdDate;

    // Constructors
    public User() {}

    public User(int userId, String username, String email, String userType, String status) {
        this.userId = userId;
```

```
    this.username = username;
    this.email = email;
    this.userType = userType;
    this.status = status;
}

public User(String username, String password, String email, String userType) {
    this.username = username;
    this.password = password;
    this.email = email;
    this.userType = userType;
    this.status = "pending";
}

// Getters and Setters
public int getUserId() {
    return userId;
}

public void setUserId(int userId) {
    this.userId = userId;
}
```

```
public String getUsername() {
    return username;
}

public void setUsername(String username) {
    this.username = username;
}

public String getPassword() {
    return password;
}

public void setPassword(String password) {
    this.password = password;
}

public String getEmail() {
    return email;
}

public void setEmail(String email) {
    this.email = email;
}
```

```
public String getUserType() {
    return userType;
}

public void setUserType(String userType) {
    this.userType = userType;
}

public String getStatus() {
    return status;
}

public void setStatus(String status) {
    this.status = status;
}

public Timestamp getCreatedDate() {
    return createdDate;
}

public void setCreatedDate(Timestamp createdDate) {
    this.createdDate = createdDate;
```

```
}

// Utility Methods
public boolean isAdmin() {
    return "admin".equalsIgnoreCase(userType);
}

public boolean isActive() {
    return "active".equalsIgnoreCase(status);
}

public boolean isPending() {
    return "pending".equalsIgnoreCase(status);
}

public boolean isBlocked() {
    return "blocked".equalsIgnoreCase(status);
}

@Override
public String toString() {
    return "User{" +
            "userId=" + userId +
```

```
            ", username='" + username + '\'' +
            ", email='" + email + '\'' +
            ", userType='" + userType + '\'' +
            ", status='" + status + '\'' +
            ", createdDate=" + createdDate +
            '}';
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;
        User user = (User) o;
        return userId == user.userId;
    }

    @Override
    public int hashCode() {
        return Integer.hashCode(userId);
    }
}
```

AuthServices.java

```
package services;
```

```
import models.User;
import java.util.*;



public class AuthService {

    // In-memory user storage: username -> User
    private Map<String, User> users;

    public AuthService() {
        users = new HashMap<>();
        // Initialize with default admin user
        users.put("admin", new User("admin", "admin123", "admin@ufc.com", "admin"));
    }

public String login(String username, String password) {
        User user = users.get(username);

        if (user == null) {
            return null; // User not found
        }

        if (!user.getPassword().equals(password)) {
```

```
        return null; // Wrong password
    }

    // Return user type ("admin", "user", or "pending")
    return user.getType();
}


public boolean register(String username, String password, String email) {
    // Check if username already exists
    if (users.containsKey(username)) {
        return false;
    }

    // Create new user with "pending" status
    User newUser = new User(username, password, email, "pending");
    users.put(username, newUser);
    return true;
}

    public List<User> getPendingUsers() {
    List<User> pendingUsers = new ArrayList<>();
    for (User user : users.values()) {
        if ("pending".equalsIgnoreCase(user.getType())) {
```

```
            pendingUsers.add(user);
        }
    }
    return pendingUsers;
}



public boolean approveUser(String username) {
    User user = users.get(username);
    if (user != null && "pending".equalsIgnoreCase(user.getType())) {
        user.setType("user"); // Change from "pending" to "user"
        return true;
    }
    return false;
}

public boolean rejectUser(String username) {
    User user = users.get(username);
    if (user != null && "pending".equalsIgnoreCase(user.getType())) {
        users.remove(username); // Remove rejected user
        return true;
    }
    return false;
}
```

```
public List<User> getAllUsers() {
    return new ArrayList<>(users.values());
}


public int getPendingUsersCount() {
    int count = 0;
    for (User user : users.values()) {
        if ("pending".equalsIgnoreCase(user.getType())) {
            count++;
        }
    }
    return count;
}


public int getActiveUsersCount() {
    int count = 0;
    for (User user : users.values()) {
        if (user.getType() != null &&
            !user.getType().equalsIgnoreCase("pending")) {
            count++;
```

```
        }
    }
    return count;
  }

  public User getUser(String username) {
    return users.get(username);
  }
}
```

REFERENCES

1. https://docs.oracle.com/en/database/oracle/oracle-database/21/jjdbc/

2.https://stackoverflow.com/questions/tagged/java+swing

3. https://docs.oracle.com/javase/tutorial/uiswing/